



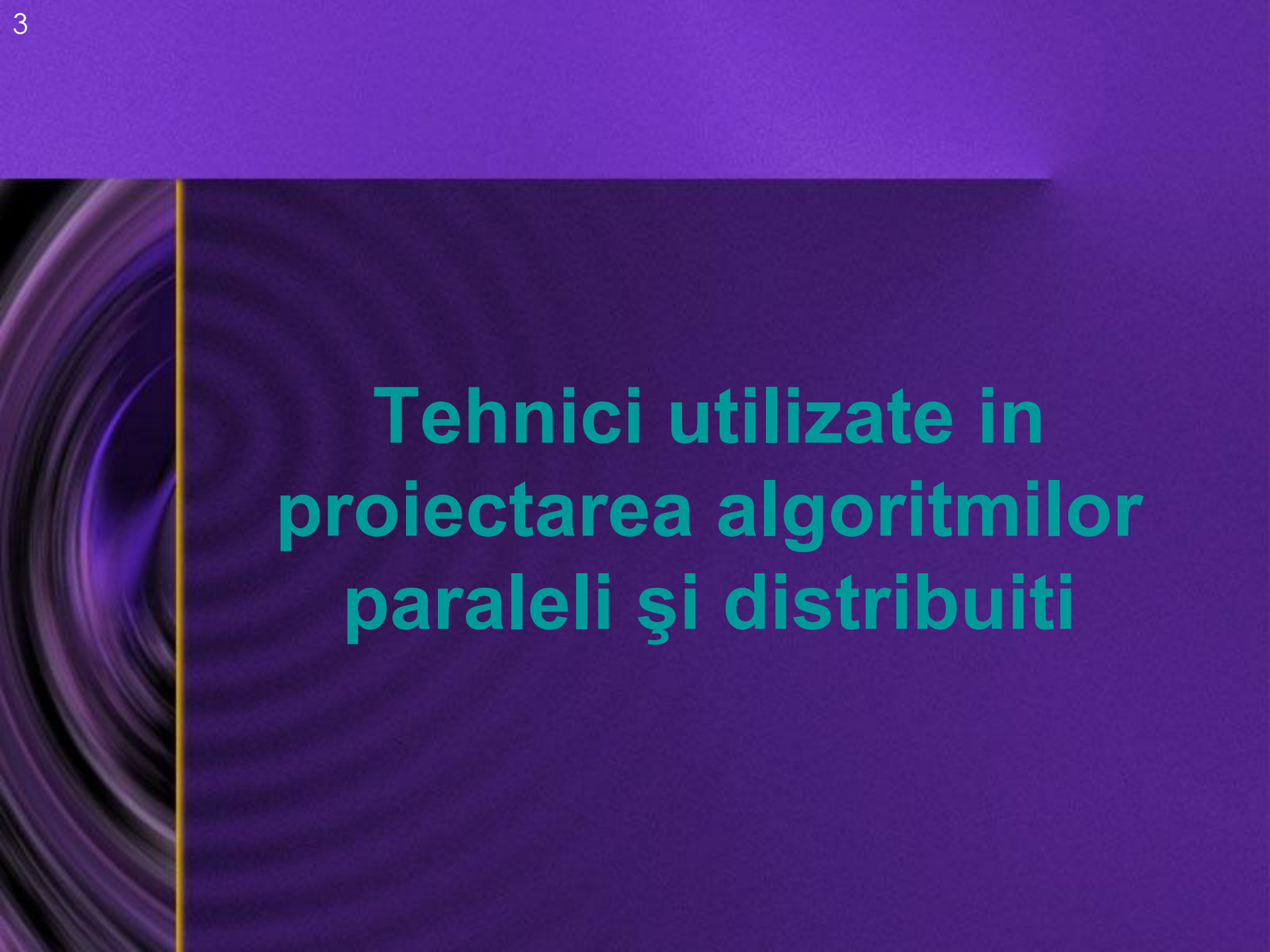
# ALGORITMI PARALELI ȘI DISTRIBUIȚI

## Curs 5

**Facultatea de Electronică, Comunicații și Calculatoare**  
**Universitatea Națională de Știință și Tehnologie Politehnica**  
**București – Centrul universitar Pitești**

# APD

1. Tehnici utilizate in proiectarea algoritmilor paraleli și distribuiti
2. Parametrii de performanta ai procesarii paralele
3. Consideratii privind proiectarea aplicatiilor paralele



# **Tehnici utilizate in proiectarea algoritmilor paraleli și distribuiți**

# Tehnici utilizate in proiectarea algoritmilor paraleli și distribuiti

Tehnicile de bază utilizate in realizarea aplicațiilor paralele sunt:

## 1. Tehnica pipeline

- datele trec de la un proces la altul;
- Exemplu: determinarea numerelor prime din setul  $1, 2, \dots, n$  cu “ciurul lui Eratostene”.

## 2. Paralelismul geometric

- se partiționează datele în submulțimi;
- Exemplu: determinarea numerelor prime între 1 și  $n$

# Tehnici utilizate în proiectarea algoritmilor paraleli și distribuiți

## 3. Paralelismul după rezultat;

- Se poate aplica în situațiile în care un program produce ca rezultat o structură de date cu mai multe elemente componente;
- Exemplu: algoritmul pentru înmulțirea a două matrice.

## 4. Ferma de procese.

- model flexibil înrudit cu paralelismul după rezultat, dar presupune, în general, crearea unui număr mai mic de procese;
- se descompune programul într-un set de activități (nu neapărat identice) care vor fi efectuate de un set de procese lucrător (*worker*);

# Tehnici utilizate în proiectarea algoritmilor paraleli și distribuiți

## Comparație:

| Caracteristică                  | Paralelism Pipeline                                  | Paralelism Geometric                                           | Paralelism după Rezultat                              | Ferma de Procese                                                  |
|---------------------------------|------------------------------------------------------|----------------------------------------------------------------|-------------------------------------------------------|-------------------------------------------------------------------|
| <b>Cum se împarte problema?</b> | Împărțirea în etape secvențiale (procesare în serie) | Împărțirea unui domeniu geometric (spațial)                    | Împărțirea în sarcini independente                    | Împărțirea în procese independente care pot rula în paralel       |
| <b>Independența sarcinilor</b>  | Sarcinile sunt dependente (etape secvențiale)        | Sarcinile pot fi dependente, dar sunt procesate pe sub-domenii | Sarcinile sunt complet independente                   | Sarcinile sunt independente                                       |
| <b>Comunicare între procese</b> | Comunicarea între etape ale pipeline-ului            | Necesită comunicare între procese pe marginea sub-domeniilor   | Comunicare pentru combinarea rezultatelor parțiale    | Comunicare pentru colectarea și combinarea rezultatelor           |
| <b>Exemple tipice</b>           | Procesare în flux continuu (video, audio, semnale)   | Simulări numerice (elemente finite, CFD)                       | Împărțirea calculului pe intervale (sortare, căutare) | Procesare batch, renderizare grafică, simulări numerice           |
| <b>Avantaje</b>                 | Eficiență în procesarea secvențială a datelor        | Scalabilitate mare în aplicațiile de simulare                  | Flexibilitate și aplicabilitate largă                 | Scalabilitate mare, eficiență în procesarea paralelă a sarcinilor |
| <b>Provocări</b>                | Dependență între etape, eficiența comunicării        | Necesitatea sincronizării datelor de la margini ↓              | Necesitatea combinării corecte a rezultatelor         | Gestionarea sarcinilor și comunicarea între procese               |

# **Parametrii de performanta ai procesării paralele**

# Parametrii de performanta ai procesarii paralele

- **Lucrul** (*work* –  $W$ ) se definește ca numărul total de operații care se execută în cadrul unui algoritm paralel. Acest parametru este dependent de dimensiunea intrării  $n$ ,  $W(n)$  și de numărul procesoarelor  $p$ ,  $W(p)$ .
- **Accelerarea paralelă** (*parallel speedup* –  $S_p$ ).

$$Acceleratia = \frac{\text{timpul secvential de executie (timpul pentru 1 proces)}}{\text{timpul paralel de executie (timpul pentru p procese)}}$$

$$1 \leq S_p = \frac{T_s}{T_p} = \frac{p}{1 + f(p-1)} \leq p$$

**Legea lui Amdahl (1967)**

# Parametrii de performanta ai procesarii paralele

Din legea lui Amdahl :

*Un număr mic de operații seriale pot limita sensibil viteza atinsă de un calculator paralel.*

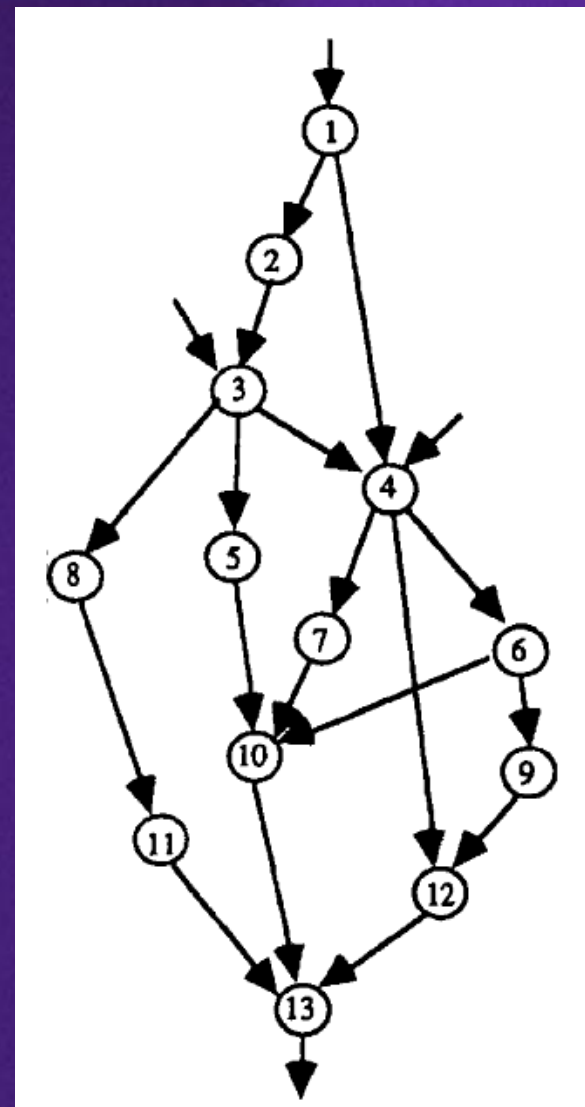
Astfel, dacă **10%** din calcul trebuie să se execute serial, atunci **viteza maximă ce poate fi atinsă este de ??, indiferent de numărul procesoarelor.**

# Parametrii de performanta ai procesarii paralele

Un **singur procesor** poate executa cele **13 noduri** in **?? unitati de timp**.

**Paralel** se poate executa, in cazul cel mai favorabil, in **?? unitati de timp**.

*Asadar, acceleratia asociata acestui graf particular **nu va putea niciodata fi mai mare de ??**, indiferent cat de multe procesoare s-ar utiliza.*



# Parametrii de performanta ai procesarii paralele

- **Eficiența paralelă (parallel efficiency –  $E_p$ )** se definește ca raportul dintre **acelerarea paralelă** și **numărul de procesoare**:

$$E_p = \frac{S_p}{p} = \frac{T_s}{pT_p}$$

- **Costul paralel (parallel cost –  $C_p$ )** se definește ca suma timpilor consumați de toate procesoarele pentru rezolvarea algoritmului. Deci, **produsul** dintre **timpul de execuție paralelă** și **numărul de procesoare**:

$$C_p = p T_p$$

**Nota:**

Deoarece în cazul unui **algoritm secvențial**,  **$p = 1$** , costul  **$C_s = T_s$** . Pornind de la această considerație, eficiența se mai poate exprima ca raportul dintre costul secvențial și costul paralel:

$$E_p = \frac{C_s}{C_p}$$

# Parametrii de performanta ai procesarii paralele

- **Redundanța (Redundancy –  $R_p$ )** se definește ca raportul dintre **lucrul total al procesoarelor paralele** și **lucrul total al unui calculator secvențial** necesar pentru rezolvarea algoritmului:

$$R_p = \frac{W(p)}{W(1)}$$

- **Utilizarea (Utilization –  $U_p$ )** se definește ca raportul dintre **lucrul total al procesoarelor paralele** și **costul paralel**:

$$U_p = \frac{W(p)}{C_p} = \frac{W(p)}{pT_p}$$

- **Calitatea (Quality –  $Q_p$ )** se definește prin relația:

$$Q_p = \frac{T_s^3}{C_p W(p)} = \frac{T_s^3}{pT_p W(p)}$$

# Parametrii de performanta ai procesarii paralele

$$U_p = R_p E_p$$

$$Q_p = E_p \frac{S_p}{R_p}$$

$$\frac{1}{p} \leq E_p \leq U_p \leq 1$$

$$1 \leq R_p \leq \frac{1}{E_p} \leq p$$

$$Q_p \leq S_p \leq p$$

# Costul suplimentar de calcul paralel

- **Costul suplimentar de calcul paralel (overhead -To)** se definește ca acea parte din costul paralel care nu intervine în cel mai rapid algoritm secvențial de rezolvare a unei probleme date:

$$T_o = C_p - T_s = p T_p - T_s$$

- **Principalele surse** care determină costul suplimentar sunt:
  - încărcarea neechilibrată a procesoarelor (load imbalance),
  - calculele suplimentare și
  - comunicația între procese



# **Consideratii privind proiectarea aplicatiilor paralele**

# Etape de realizare a aplicatiilor paralele

Principalele etape de realizare a unui program paralel sunt:

1. **Identificarea porțiunilor aplicației care pot fi executate în parallel.**
2. **Descompunerea aplicației în mai multe părți, plecând de la:**
  - domeniul datelor;
  - funcționalitatea aplicației.
3. **Faza de proiectare**
4. **Compilarea, testarea și depanarea aplicației.**
5. **Optimizarea aplicației**

# Etape de realizare a aplicatiilor paralele

Proiectarea aplicatiilor paralele trebuie sa tina cont de

4 factori principali:

1. **Încărcarea echilibrată a procesoarelor (load balancing)**
2. **Granularitatea**
  - a. **Operație**
  - b. **Instrucțiune**
  - c. **Iterație**
  - d. **Subprogram**

# Etape de realizare a aplicațiilor paralele

3. **Operațiile de I/O** reduc, în general, gradul de paralelism al unei aplicații.
4. **Configurația arhitecturii destinație.** Obținerea unor performanțe bune ale unei aplicații paralele presupune utilizarea unor algoritmi adaptați arhitecturii respective. Pentru aceasta, este nevoie ca programatorul să ia în considerare factori ca:
  - numărul procesoarelor;
  - caracteristicile memoriei;
  - încărcarea sistemului.